迭代评估报告2

评估日期：7 月 24 日

组号	5	项目名称	Arxiv-PaperMate
迭代名称	技术原型迭代	实际起止日期	7月 14 日 - 7 月 24 日

任务达成情况：

1. 制定本迭代的《迭代计划》

2. 技术方案设计

- 选择架构风格，确定语言、框架、工具
- 设计多个架构视图
- 设计关键算法(若需要)
- 撰写和评审软件架构文档
- 选定或撰写编程规范(如google Java编程规范)

3. 实现技术原型

- 搭建软件架构
- 实现关键算法或技术
- 实现优先级高的需求

4. 测试技术原型

- 测试是否达到需求中预定的技术要求，并根据测试结果进行改进

5. 编写《迭代评估报告》

- 评审和测试记录，开发总结

质量情况：（执行了哪些评审和测试？评审和测试的结果如何？）

1. 执行需求一致性评审，已经基本实现需求分析的要求
2. 数据模型评审，通过数据模型评审确定了数据模型
3. 端到端冒烟测试，(主要功能合并都进行了冒烟测试)，主要功能无问题
4. 单元测试，测试独立可验证的业务逻辑都通过

- arXiv 返回结果解析；
- 论文去重逻辑；
- 搜索参数校验；
- PDF 文本清洗；

- 文本分块；
- 相似度结果排序；
- 权限判断；
- 问答出处绑定；

5. 异常测试，测试异常输入等，测试结果均正常

- 输入为空或问题过短；
- 比较论文时选择未解析的论文；
- 知识库中无相关论文；
- LLM 服务调用失败；

6. 检索与问答质量测试
7. 性能基线测试
8. 安全测试

问题、变更和返工：（遇到的问题、发生的变更、是否需要返工等）

1. 相当一部分页面（如对比页面，用户设置页面）为静态页面，无法反映真实的内容

- 解决方式：将相关的mock改为真实的API

2. 论文pdf解析格式不稳定，多次 parse right 成功率低

- 解决方式: 数据清洗、停止自动解析、逐步优化 pipeline

3. Embedding 问题，RAG 相关功能不稳定

- 解决方式: 多轮 hotfix

4. Alembic migration 冲突，多人同时加 migration 文件

- 解决方式: 集中解决 revision 冲突

经验和教训：

经验：

1. 严格 PR Review 流程保证主干稳定

- 我们提交历史中所有合并都走了 PR（174 个 commit 都是 PR merge 而非直接 push main），commit图全是清晰的分支合并线。这意味着主干永远可部署，没人能破坏别人的工作。

2. 多人并行开发 + 功能分片效率高

- 每个人负责独立的页面/模块，功能分片粒度合理，避免了大量互相等待。

3. 渐进式集成降低耦合风险

- 前端先用 Mock 数据开发，后端独立推进，然后再切换真实API。这样前后端不用等对方完成就能开工，最后一步联调即可。

教训：

1. 论文 PDF 解析严重低估了难度

   从 parse debug → parse right (×3) → before no parse → better parse -- datacleaning →free some bugs, stop auto-parsing → stable parse，前后跨了 3天多才基本稳定。问题不在代码质量，而在于没有提前验证核心技术的可行性——PDF解析（尤其是学术论文的排版多样性）远比想象中难，应该先做技术预研/原型验证再分配资源。

2. Merge Conflict 频率过高

   根源是团队没有约定工作策略：

- 多人同时间段修改同一模块（如 User 相关功能 7/20 一天出现 3 个相关 PR 碰到一起）
- 没有约定谁来维护 migration 版本的次序
- 缺乏 git 工作流规范（如谁 rebase 谁 merge、分支与 main 同步频率）

3. Alembic migration 没有串行化管理

   7/17 集中爆发 migration revision冲突。如果一开始就定一个规则——"每天只有一个人负责生成 migration 文件"或"生成migration 前先跑 alembic upgradehead"——可以完全避免。越到后期、多人改越多表，migration 冲突的修复成本越高。